Comparing Verification Formulations for Neural Code Repair: A Controlled Study across Model Families

Anonymous

Abstract

Execution-free neural verification of code repairs can be formulated as per-test binary classification (CLS), execution output generation (CWM), or trajectory-level resolved/unresolved scoring (TRAJ). Despite the proliferation of these approaches, no controlled comparison exists that holds model architecture, training data, and compute constant. We present a systematic study comparing all three formulations on two model families (Qwen3 4B/8B and DeepSeek-Coder 6.7B) at both function-level and repository-level complexity, using identical LoRA configurations and multi-seed protocols. At function-level, all three formulations achieve statistically equivalent accuracy (two one-sided tests [TOST] equivalence p=0.007 within $\pm$2 percentage points). At repository-level, only per-test classification on Qwen3 yields results substantially above the majority baseline (83.6%–85.6%), while generation-based CWM collapses on both model families with unparseable output rates exceeding 75%. We further identify significant intra-problem correlation in per-test predictions (intraclass correlation coefficient [ICC] = 0.19, design effect = 3.97), which means that standard confidence intervals underestimate variance by a factor of four. Our results provide practical guidance for verification system design and surface methodological pitfalls in per-test evaluation.

1 Introduction

Automated code repair systems generate candidate patches for software bugs, but verifying whether a patch is correct without executing it remains an open challenge. Execution-based verification (running the test suite) is the gold standard, yet it requires sandboxed environments, dependency resolution, and significant compute time. As repair systems scale to process hundreds of candidates per bug, execution-free neural verifiers become essential for efficient filtering.

Three distinct formulations have emerged for this verification task. Per-test classification (CLS) treats each test case independently and predicts a binary pass/fail label. Execution output generation, as used in Code World Models (CWM), predicts the expected execution output of each test and compares it against the ground truth. Trajectory-level scoring (TRAJ) considers all tests jointly and predicts a single resolved/unresolved label for the entire repair attempt. Each formulation makes different assumptions about what granularity of feedback is useful, and each carries distinct implementation costs.

Despite the practical importance of this design choice, no prior work provides a controlled comparison. Existing studies employ different models, datasets, training procedures, and evaluation metrics, so performance differences cannot be attributed to the formulation itself. A practitioner choosing among these approaches must rely on intuition or ad-hoc comparisons.

Beyond formulation choice, the evaluation methodology for per-test verifiers has received little scrutiny. When multiple tests belong to the same problem instance, predictions for those tests may be correlated, because they share the same code context and the same model representation. If this intra-problem dependence is substantial, standard per-test accuracy metrics will overstate statistical precision.

We address these gaps with a controlled experimental study. We train CLS, CWM, and TRAJ verifiers using the same base models (Qwen3-4B, Qwen3-8B, DeepSeek-Coder-6.7B), the same LoRA configuration, the same training data, and the same evaluation protocol. We evaluate at two complexity levels (function-level and repository-level) and replicate all core experiments across multiple random seeds. Our key findings are:

1. **Function-level formulation equivalence.** CLS, CWM, and TRAJ achieve statistically equivalent accuracy at function-level across both model families (TOST p=0.007, $\pm$2pp margin). Formulation choice does not affect accuracy when execution outputs are short and structured (Section 4.1).

2. **Repository-level divergence.** At repository-level, only CLS on Qwen3 exceeds the majority baseline by a meaningful margin (83.6%–85.6%). CWM collapses on both model families with unparseable rates of 53%–93%, and DeepSeek-Coder CLS barely matches the majority baseline. Generation-based verification is impractical at 4–8B LoRA scale when outputs are long (Section 4.6).

3. **Per-test diagnostic granularity.** Per-test CLS provides failure localization (93.6% fail recall) with per-mutation-type diagnostics, at 6–9$\times$ lower inference cost than trajectory-level approaches, without sacrificing overall accuracy (Section 4.3).

4. **Block correlation in evaluation.** Intra-problem correlation (ICC = 0.19, design effect = 3.97) means that standard confidence intervals for per-test accuracy underestimate variance by approximately 4$\times$. This finding applies to any evaluation that treats per-test predictions as independent (Section 4.5).

2 Related Work

Execution-free code verification. CodeT [1] uses dual execution agreement for candidate ranking. LEVER [8] trains a verifier on execution results to rerank code generation candidates. These works focus on candidate selection rather than per-test diagnostics and do not compare formulations under controlled conditions.

Code World Models. Code World Models [2] train language models to predict execution outputs, using these predictions as a proxy for actual execution. The original approach uses 32B+ models with mid-training on execution traces. Our experiments compare the CWM formulation at the smaller 4–8B LoRA scale and reveal that the output parsing requirement becomes a critical bottleneck at repository-level complexity.

Trajectory-level verification. SWE-RewardModel [12] and related approaches predict a single resolved/unresolved label for an entire repair trajectory. This formulation avoids per-test prediction but sacrifices diagnostic granularity, providing only a coarse binary signal about overall success.

Process vs. outcome supervision. The distinction between per-test and trajectory-level verification parallels the process vs. outcome supervision debate in mathematical reasoning [7, 10]. Process supervision provides step-level feedback while outcome supervision evaluates only the final answer. Our CLS vs. TRAJ comparison maps to this framework, with per-test verification analogous to process-level feedback.

Predictive mutation testing. The software engineering community has studied predicting test outcomes under code mutations [11]. Per-test binary classification for verification is a special case of predictive mutation testing. Our work extends this line by comparing classification against generation-based and trajectory-level alternatives.

3 Method

3.1 Verification Formulations

Given a code repair candidate and associated test cases, we study three formulations.

Table 1: Function-level verification accuracy (%). Mean ± standard deviation across seeds reported where ≥3 seeds available. Majority baseline: 51.75%.

Model	Size	CLS	CWM	TRAJ	Seeds
Qwen3	4B	**88.43**±0.73	87.19±0.37	86.46±1.01	5/5/4
Qwen3	8B	**89.39**±1.09	86.96±0.72	–	3/3/–
DeepSeek	6.7B	**86.85**	86.83	84.16	2/2/2
0-shot Qwen3-4B			68.73		–
0-shot DeepSeek-6.7B			63.02		–
Majority baseline			51.75		–

Per-test classification (CLS). The input consists of the source code (original or patched) and a single test case. The model predicts a binary label: `pass` or `fail`. Each test case is processed independently, producing a vector of per-test predictions.

Execution output generation (CWM). The input is identical to CLS. Instead of a binary label, the model generates the expected execution output of the test case. The generated output is then compared against the ground-truth output to determine pass or fail. This formulation requires the model to produce parseable, well-formatted outputs.

Trajectory-level scoring (TRAJ). The input consists of the full repair context (original code, patched code, and all test cases concatenated). The model predicts a single binary label: `resolved` or `unresolved`. This formulation aggregates information across all tests into one holistic prediction.

3.2 Datasets

Function-level. We construct a function-level dataset derived from CRUXEval [3]. Each instance pairs a Python function with a test assertion. We apply systematic mutations (return value changes, off-by-one errors, boundary condition modifications) to generate fail cases. The dataset is balanced (majority baseline: 51.75%).

Repository-level. For repository-level evaluation, we use a subset derived from SWE-bench [6]. Each instance contains the repository context, a candidate patch, and associated test cases. The dataset is imbalanced toward pass (majority baseline: 72.96%), reflecting real-world distributions where most tests pass even for incorrect patches.

3.3 Training Protocol

We use LoRA fine-tuning [5] on two model families: Qwen3 [9] (4B and 8B) and DeepSeek-Coder [4] (6.7B). All models use the same LoRA rank, learning rate, batch size, and training schedule. For each formulation and model combination, we train with multiple random seeds (up to 5 for core comparisons) and report mean ± standard deviation. This multi-seed protocol enables TOST equivalence testing and guards against seed-dependent conclusions.

4 Experiments

4.1 Function-Level Formulation Comparison

Table 1 presents the function-level results across all formulations and model families. The central finding is that CLS, CWM, and TRAJ achieve comparable accuracy, with no formulation showing a consistent advantage.

Table 2: Multi-seed statistics for function-level Qwen3-4B. The TOST procedure tests whether the CLS–CWM gap falls within ±2pp.

Form.	Seeds	Mean (%)	Std	Gap vs. CLS (pp)	TOST p
CLS	5	88.43	0.73	–	–
CWM	5	87.19	0.37	−1.24	0.007
TRAJ	4	86.46	1.01	−1.97	–

Table 3: Per-test failure prediction quality (func-level). Overall accuracy: 87.51%.

Metric	Precision (%)	Recall (%)	F1 (%)
Fail class (overall)	82.75	93.64	87.86
Pass class (overall)	–	–	87.31
Accuracy by mutation type (%)			
return_value		97.26	
off_by_one		82.56	

For the core Qwen3-4B comparison (5 seeds for CLS and CWM, 4 for TRAJ), we apply a TOST equivalence procedure with a ±2 percentage point margin. The test confirms statistical equivalence between CLS and CWM (p=0.007). Table 2 summarizes the multi-seed statistics.

The CLS–TRAJ gap (1.97pp) falls at the boundary of the equivalence margin. TRAJ shows higher variance across seeds (std = 1.01 vs. 0.73 for CLS and 0.37 for CWM), consistent with problem-level randomness introduced by trajectory aggregation.

Cross-model validation on DeepSeek-Coder confirms this pattern. The CLS–CWM gap is 0.02pp (86.85% vs. 86.83%), even smaller than on Qwen3. CWM achieves 0% unparseable predictions at function-level on both model families, because execution outputs at this complexity level are short and structurally simple (typically fewer than 50 tokens). The CLS–TRAJ gap on DeepSeek (2.69pp) is slightly larger than on Qwen3 (1.97pp), possibly reflecting DeepSeek-Coder's weaker instruction-following in the trajectory aggregation setting.

Zero-shot baselines (68.73% for Qwen3-4B, 63.02% for DeepSeek) confirm that task-specific fine-tuning is essential, with fine-tuned models outperforming zero-shot by 18–25 percentage points. We also evaluate larger zero-shot models as judges: Qwen3-8B per-test accuracy is 64.4% and Qwen3-32B reaches 71.69%, both well below the fine-tuned 4B model. Zero-shot trajectory-level evaluation achieves 99.1% accuracy, but this matches the trajectory majority baseline exactly (99.1% of instances are "unresolved"), exposing the futility of zero-shot trajectory evaluation under extreme class imbalance.

4.2 Model Size Scaling

Scaling from 4B to 8B parameters reveals an asymmetry between formulations. CLS improves from 88.43% to 89.39% (+0.96pp), while CWM is essentially unchanged (87.19% to 86.96%, −0.23pp). At 8B, the CLS–CWM gap widens to 2.43pp, exceeding the multi-seed noise range observed at 4B.

This asymmetry points to different scaling bottlenecks. CLS benefits from additional model capacity because it is a binary classification task where better representations improve the decision boundary for ambiguous cases. CWM requires exact output generation, and once the model reliably generates correct output format (achievable at 4B for function-level), additional parameters provide diminishing returns. The ceiling is set by the exact-match constraint, not by representation quality.

4.3 Per-Test Diagnostic Quality

Per-test verification provides fine-grained failure localization that trajectory-level scoring cannot offer. Table 3 reports precision, recall, and F1 for the fail class, alongside accuracy stratified by mutation type.

Table 4: Oracle advantage of per-test over trajectory-level predictions, stratified by number of failing tests (T). "Block adv." measures information gain from knowing *which* tests fail.

T (failing tests)	% of instances	Block advantage
T=1	68.7	−0.09
T=2	19.1	31.69
T=3–5	–	40.74
$T \geq 6$	–	40.06
Overall	100.0	10.93

The model achieves 93.64% recall for failure detection, capturing the vast majority of true failures, while precision of 82.75% indicates that roughly 17% of predicted failures are false alarms. In a diagnostic setting where the cost of missing a failure exceeds the cost of investigating a false alarm, this recall-biased profile is appropriate.

Mutation type analysis reveals that return value mutations are easiest to detect (97.26% accuracy) because they produce clearly different outputs. Off-by-one errors are hardest (82.56%), as the model must reason about subtle boundary conditions. This per-mutation diagnostic information is available only through per-test formulations; trajectory-level scoring cannot identify which tests fail or categorize the nature of failures.

We also evaluate per-test consistency between CLS and TRAJ: 79.94% of per-test CLS predictions agree with the corresponding TRAJ verdict. The 20% disagreement confirms that trajectory-level aggregation masks per-test level nuance, and specific failure information is lost.

4.4 Oracle Advantage of Per-Test Granularity

To quantify the theoretical benefit of per-test predictions, we compute the oracle advantage: the improvement in downstream selection accuracy when per-test predictions are available instead of only trajectory-level labels. We stratify by the number of failing tests T (Table 4).

The advantage concentrates in instances with $T \geq 2$ failing tests. When only one test fails (T=1, 68.7% of instances), per-test and trajectory-level information are equivalent: a single failing test fully determines the trajectory outcome. The T=2 stratum contributes 55.3% of the total advantage despite comprising only 19.1% of instances. For $T \geq 3$, the advantage exceeds 40 points, but such instances are rare.

We extend this analysis to include pass-to-pass (P2P) regression tests. At a 5% regression rate, the total advantage is negligible (−0.005, not significant). At 10%, it becomes marginally positive (0.083). P2P tests contribute meaningful diagnostic information only when regression rates reach 10% or higher, which is uncommon for high-quality repairs. The practical implication is that per-test granularity primarily benefits fail-to-pass (F2P) test analysis.

4.5 Block Correlation in Per-Test Evaluation

Per-test evaluation implicitly assumes that predictions for different tests within the same problem are independent. We test this assumption using the intraclass correlation coefficient (ICC) computed across problems (Table 5).

The ICC of 0.191 ($p < 0.001$) yields a design effect of 3.97. The effective sample size for per-test metrics is approximately one quarter of the nominal test count. Standard confidence intervals computed under the i.i.d. assumption underestimate true variance by a factor of four.

The observed all-correct rate provides an intuitive illustration: 46 of 114 problems (40.4%) have every per-test prediction correct, compared to 16.35 (14.3%) expected under independence. The ratio of 2.81 confirms that model errors cluster within problems. When the model fails on one test, it tends to

Table 5: Block correlation analysis of per-test predictions. Design effect indicates the ratio of true variance to variance under the i.i.d. assumption.

Statistic	Value
ICC	0.191
p-value	< 0.001
Design effect	3.97
Observed all-correct problems	46
Expected all-correct (i.i.d.)	16.35
Observed / expected ratio	2.81

Table 6: Repository-level verification results. Majority baseline: 72.96%. "Unp." = unparseable output rate (%). Results below the majority baseline are marked with †.

Model	Form.	Acc (%)	Fail F1	Unp. (%)
Qwen3-4B	CLS	83.63	61.67	–
Qwen3-8B	CLS	**85.60**	–	–
Qwen3	CWM	$\leq$69.91†	–	53–93
DeepSeek-6.7B	CLS (s42)	71.50†	28.30	$<$1
DeepSeek-6.7B	CLS (s123)	74.93	22.84	–
DeepSeek-6.7B	CWM (s42)	52.30†	18.50	82.0
DeepSeek-6.7B	TRAJ	75.35$\pm$4.89	–	–
0-shot Qwen3-4B		49.90†	–	–
0-shot DeepSeek-6.7B		48.54†	–	–
Majority baseline		72.96	–	–

fail on other tests in the same problem, because the errors stem from a shared misunderstanding of the underlying code logic.

This finding has direct implications for evaluation methodology. Per-test accuracy alone can produce overconfident comparisons between systems. We recommend that future work report problem-level accuracy alongside per-test accuracy, or adjust confidence intervals using the design effect.

4.6 Repository-Level Case Study

Table 6 extends the comparison to repository-level complexity. Unlike the function-level setting, repository-level results vary dramatically across formulations and model families.

Qwen3 CLS. Qwen3-4B CLS achieves 83.63% test accuracy, exceeding the majority baseline by 10.67pp. Scaling to 8B yields 85.60% (best of two seeds: 84.79% for seed 42, 85.60% for seed 123). However, two issues weaken confidence in these results. First, the validation-test gap of 6.44pp for the 4B model (val 90.07%, test 83.63%) suggests overfitting or distribution shift. Second, the optimal checkpoints differ substantially across seeds (step 400 vs. step 3000 for 8B), indicating training instability at repository-level. Fail F1 of 61.67% for the 4B model (compared to 87%+ at function-level) reflects the difficulty of repository-level classification.

CWM collapse on both model families. CWM fails at repository-level on both Qwen3 and DeepSeek-Coder. Qwen3 CWM produces unparseable outputs in 53%–93% of cases across configurations (v10: 93% unparseable, v11: 75.2%, v12: 53.7%), yielding at best 69.91% accuracy, below the majority baseline. DeepSeek CWM achieves only 52.3% accuracy (seed 42, epoch 3) with 82% unparseable outputs. An earlier epoch checkpoint (epoch 2: 56.8%, 76% unparseable) and a second seed (seed 123: approximately 55%, 80% unparseable, training terminated early) show marginal improvement, confirming that

the failure is not merely overfitting.

The contrast with function-level is instructive. Both model families achieve 0% unparseable at function-level but 53%–93% at repository-level. This rules out a purely tokenizer-specific explanation. Instead, the CWM formulation faces a fundamental capacity limitation: repository-level execution outputs are long (often hundreds of tokens) and structurally complex, exceeding what 4–8B LoRA models can generate reliably.

DeepSeek CLS. DeepSeek-Coder CLS achieves 71.50% (seed 42) and 74.93% (seed 123), only marginally above the 72.96% majority baseline. Fail F1 of 28.30% and 22.84% reveals near-complete collapse to majority-class prediction: fail recall is below 14% for seed 123, meaning 86% of true failures are missed. The likely cause is input truncation at 2,048 tokens, as repository-level inputs average approximately 5,000 tokens.

DeepSeek TRAJ. The only viable DeepSeek repository-level result comes from TRAJ, which achieves 75.35% $\pm$ 4.89% across 6 seeds on a test set of n=48 problems. This exceeds the majority baseline by 21pp, but the small test set and high cross-seed variance (range: 66.67%–81.25%) limit the reliability of the estimate.

Zero-shot baselines. Zero-shot performance is near random for both models (49.9% and 48.5%), far below the 73% majority baseline. Repository-level verification without fine-tuning is entirely infeasible at these model scales.

4.7 Ablations and Negative Results

Trace-then-judge. We explored a chain-of-thought variant where the model first generates an execution trace and then predicts pass/fail based on that trace. This approach failed: the Qwen3-4B LoRA model could not learn to produce meaningful traces and instead generated template text before defaulting to a label. Explicit execution reasoning appears to require either larger models or supervised trace data.

Self-consistency via majority voting. K=3 majority voting on CWM predictions decreased accuracy from 87.51% to 86.83% ($-$0.69pp), with 28 correct-to-incorrect flips versus 15 incorrect-to-correct flips (net $-$13). Temperature sampling introduces noise on high-confidence classification decisions without the diversity benefit observed in generation tasks. Wall-clock time increased by 4$\times$. Self-consistency does not benefit per-test verification at this accuracy level.

Downstream re-repair. We conducted a preliminary evaluation (n=44) of whether per-test diagnostic predictions improve a repair agent's pass@1 rate. Control (no diagnostic): 41.0%, CLS diagnostic: 42.5%, oracle diagnostic: 44.0%, random diagnostic: 38.0%. The 1.5pp improvement is not statistically significant (p=0.42). The repair agent extracts sufficient diagnostic signal from test source code and error messages directly; per-test predictions at 87.5% accuracy provide insufficient marginal information. More effective integration of per-test diagnostics into repair pipelines is left for future work.

5 Discussion

Practical guidance. For function-level verification at 4–8B scale, formulation choice is a design decision rather than a performance differentiator. CLS is the simplest to implement and requires no output parsing. For repository-level verification, CLS is the only viable formulation in our experiments, and only on Qwen3 with sufficient sequence length. Practitioners should prefer per-test CLS for its simplicity, diagnostic capability, and 6–9$\times$ inference cost advantage over trajectory-level approaches.

The CWM scalability barrier. CWM faces a fundamental scalability problem. At function-level, outputs are short enough (fewer than 50 tokens) that 4B models generate them reliably, and the formulation matches CLS in accuracy. At repository-level, outputs span hundreds of tokens with complex formatting, and generation quality degrades sharply. Both model families exhibit this pattern, so the barrier is complexity-driven rather than model-specific. Larger models or execution-aware pretraining may help, but at 4–8B LoRA scale, CWM is not a practical choice for complex verification tasks.

Block correlation and evaluation practice. The design effect of 3.97 should inform future evaluation protocols. When per-test predictions cluster by problem, reporting the number of test predictions as the sample size inflates statistical power. We recommend reporting both per-test and per-problem accuracy and computing confidence intervals that account for cluster structure.

Limitations. (1) We compare only 4–8B LoRA models; results may differ at larger scales or with full fine-tuning. (2) Repository-level CLS results are limited to a single model family (Qwen3); cross-family generalization at repository-level is not established. (3) We could not demonstrate downstream utility of per-test diagnostics for re-repair (Section 4.7). (4) The repository-level test set (48–114 problems) is small, limiting statistical power for fine-grained comparisons. (5) Our CWM comparison uses smaller models and less specialized training than the original Code World Models work, which may understate CWM's potential at larger scales.

6 Conclusion

We present a controlled comparison of three execution-free verification formulations for neural code repair, evaluated across two model families and two complexity levels with multi-seed replication. At function-level, all three formulations are statistically equivalent (TOST p=0.007), and formulation choice reduces to implementation preference. At repository-level, the formulations diverge sharply: only per-test classification on Qwen3 exceeds the majority baseline, while CWM collapses on both model families. We identify intra-problem block correlation (ICC = 0.19, design effect = 3.97) as a previously uncharacterized methodological concern for per-test evaluation. These findings offer actionable guidance: at small model scales, per-test classification is the robust default, and evaluation should account for intra-problem prediction dependence.

References

[1] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen. CodeT: Code generation with generated tests. In *Proc. ICLR*, 2023.

[2] Y. Ding, J. Peng, M. Min, G. Kaiser, J. Mao, and B. Ray. Traced is worth more words: Enhancing code generation with code execution results. *arXiv preprint arXiv:2402.02034*, 2024.

[3] A. Gu, B. Rozière, H. Leather, A. Solar-Lezama, G. Synnaeve, and S. I. Wang. CRUXEval: A benchmark for code reasoning, understanding and execution. In *Proc. ICML*, 2024.

[4] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li, F. Luo, Y. Xiong, and W. Liang. DeepSeek-Coder: When the large language model meets programming. *arXiv preprint arXiv:2401.14196*, 2024.

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *Proc. ICLR*, 2022.

[6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proc. ICLR*, 2024.

[7] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. In *Proc. ICLR*, 2024.

[8] A. Ni, S. Iyer, D. Radev, V. Stoyanov, W.-t. Yih, S. I. Wang, and X. V. Lin. LEVER: Learning to verify language-to-code generation with execution. In *Proc. ICML*, 2023.

[9] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[10] J. Uesato, N. Kushman, R. Kumar, F. Song, N. Siegel, L. Wang, A. Creswell, G. Irving, and I. Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.

[11] J. Zhang, L. Zhang, M. Harman, D. Hao, Y. Jia, and L. Zhang. Predictive mutation testing. *IEEE Trans. Software Eng.*, 45(9):898–918, 2019.

[12] Y. Zhang et al. SWE-Reward: Automated verification for software engineering tasks. *arXiv preprint*, 2025.